LLD

Low-Level Design

Module-level design, schemas, and interaction contracts

Document	LLD
Version	1.0
Status	Approved
Owner	Swiggy Nexus AI Team
Audience	Engineering, Product, Design, Reviewers
Last Updated	July 2026

This document is part of the Swiggy Nexus AI reference implementation, an AI-powered personal commerce assistant built on the Swiggy MCP. It captures decisions, requirements, and design intent for engineers, reviewers, and stakeholders.

1. Agent State Machine

The agent is implemented as a LangGraph with the following nodes: **router**, **planner**, **tool_selector**, **mcp_executor**, **critic**, **synthesizer**. Edges are conditional on structured outputs from each node.

Node	Input	Output
router	raw message	intent: food|instamart|dineout|chat
planner	intent + context	ordered plan of tool calls
tool_selector	plan step	MCP tool + params
mcp_executor	tool + params	typed tool result
critic	tool results	continue | refine | done
synthesizer	final state	assistant message + cart draft

2. Key Data Models (Pydantic)

```
class UserPreferences(BaseModel):
    diet: Literal['veg','non-veg','vegan','jain'] = 'non-veg'
    allergies: list[str] = []
    price_band: tuple[int, int] = (150, 1500)
    palate_vector: list[float] = Field(default_factory=list)

class CartDraft(BaseModel):
    vertical: Literal['food','instamart','dineout']
    items: list[CartItem]
    coupon_code: str | None = None
    total_paise: int
    expires_at: datetime
```

3. Database Schema (Selected Tables)

Table	Column	Type	Notes
users	id	uuid PK	gen_random_uuid()
users	email	citext UNIQUE	case-insensitive
conversations	id, user_id, summary	uuid, uuid, text	summary rolled up hourly
messages	id, conv_id, role, content, tokens, cost_paise	various	append-only
orders	id, vertical, status, total_paise	various	state machine enforced
memory_chunks	id, user_id, embedding, text, decay	vector(1536)	HNSW index

4. MCP Tool Contract

```
POST /mcp/food.search_restaurants
{ "lat": 12.97, "lon": 77.59, "cuisine": ["north_indian"],
  "veg_only": false, "max_eta_min": 35, "limit": 12 }
```

Response is a typed list of RestaurantCandidate objects; all fields validated by the MCP client before returning to the agent.

5. Concurrency & Retry

- Bulk MCP calls issued via asyncio.gather with per-call timeout of 3.5s.
- Retry policy: 3 attempts, exponential backoff (base 200ms, cap 2s), jitter enabled.
- Circuit breaker: opens after 5 failures / 30s, half-open probe every 15s.
- Idempotency-Key header on all order-mutating MCP calls.

6. Caching Strategy

Layer	Key	TTL
Menu cache	restaurant_id	5 min
Coupon cache	user_id + vertical	10 min
LLM response	sha256(prompt+model)	24 hours (safe intents only)
Embedding cache	sha256(text)	30 days

7. Sequence: Confirm & Place Order

Step	Actor	Action
1	User	POST /v1/orders/confirm with cart_draft_id
2	API	Load draft; verify user, price parity, expiry
3	Agent	Replay plan in dry-run; validate against MCP schema
4	MCP	place_order with Idempotency-Key
5	Swiggy	Order accepted; return order_id + ETA
6	Webhook	Status updates → Nexus → SSE push to client
7	Memory	Persist episodic trace + pantry deltas

8. Error Model

Error	HTTP	Client Behavior
INVALID_INPUT	400	Show inline validation
AUTH_REQUIRED	401	Redirect to OAuth

Error	HTTP	Client Behavior
RATE_LIMITED	429	Retry with Retry-After
MCP_UNAVAILABLE	503	Show degraded state, allow manual retry
INTERNAL	500	Toast + capture in Sentry

9. Testing Hooks

- Deterministic mode: seed RNG + record LLM responses via VCR.
- MCP replay: bundled fixtures per vertical for offline dev.
- Feature flags via LaunchDarkly for gradual rollout of agent behaviors.